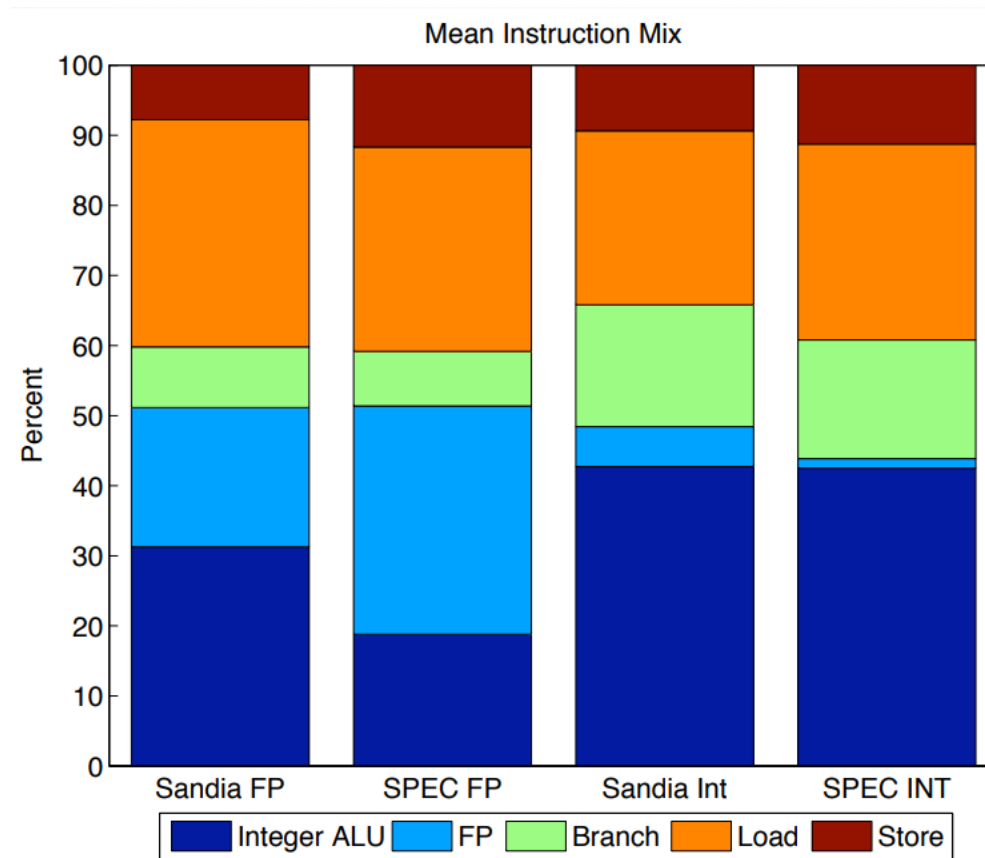


Profiling

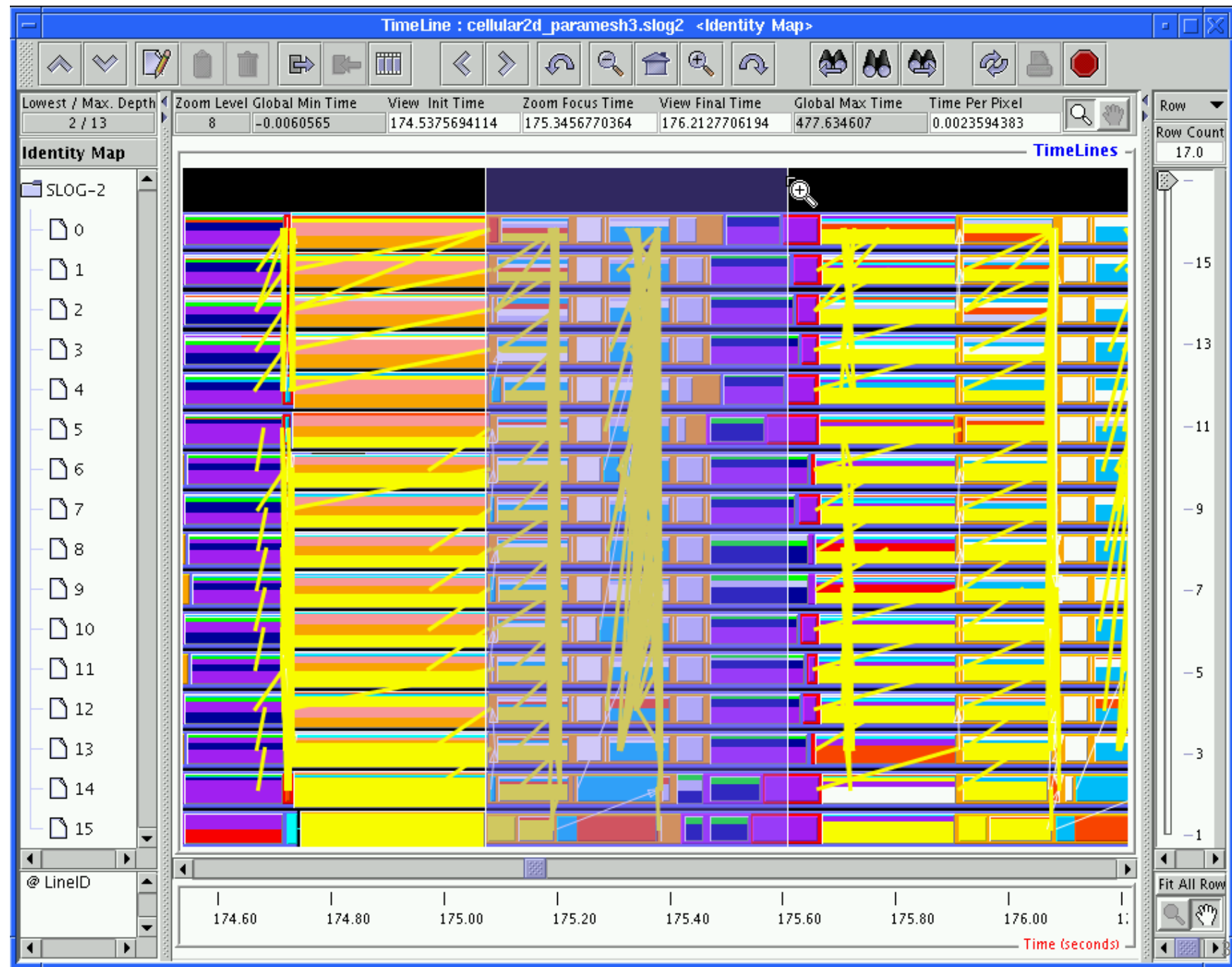
Lecture 25
April 15, 2026

Analyzing CPU Performance – Example

Mean Instruction Mix of Different Workloads

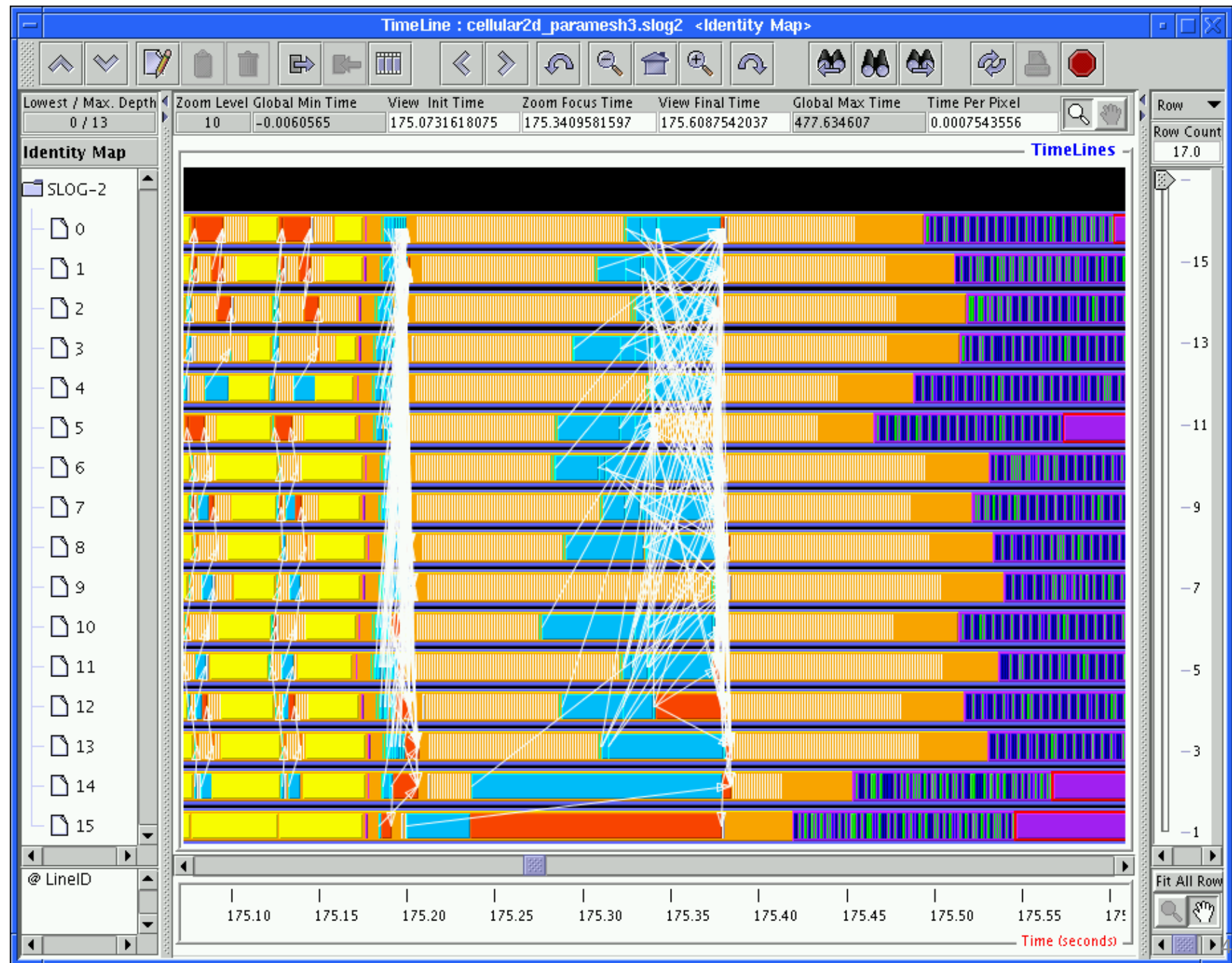


Reference: **On the Memory Access Patterns of Supercomputer Applications: Benchmark Selection and Its Implications**, Murphy and Kogge, IEEE Transactions on Computers, 2007



<https://www.mcs.anl.gov/research/projects/perfvis/software/viewers/>

Zoomed-in view from the selection window on the previous slide



Performance Analysis

- Identify hot spots
- Identify where resources are being used inefficiently
- Difference between peak and average performance
- What matters is proportion of events
 - Floating point arithmetic vs. memory operations
- Performance analysis tools must collect and present relevant information on application performance in a scalable manner
 - Enable developers to easily identify and determine the causes of performance bottlenecks
 - Crucial to be not immersed in sea of data

Profiling

- Elapsed time for the program and procedures
- Collection of statistical summaries of various performance metrics broken down by program entities such as routines and nested loops
- Performance metrics include time as well as hardware counter metrics such as operation counts and cache and memory event counts
- Profiling can identify regions of a program that are consuming the most resources

Tracing

- Timestamp attached with each event
- Collection of a timestamped sequence of events
 - Entering and exiting program regions
 - Sending and receiving messages.
- Can help identify the causes of performance problems
- Event traces record the temporal and spatial relationships between individual runtime events
- Cons: Huge space requirement

Performance Counters

Operation count data - Hardware

- Number of cache misses
- Number of loads/stores
- Main memory stalls

Operation count data - Network

- Number of packets sent and received
- Number of packets waiting to be sent

Profiling

- Source instrumentation
 - Know the location of bottleneck apriori
 - Huge endeavor to instrument every procedure in a large application
- Compile-time instrumentation
- Run-time instrumentation

Source Code Instrumentation

```
start = MPI_Wtime()
```

```
...
```

```
...
```

```
...
```

```
end = MPI_Wtime()
```

Source Code Instrumentation

```
/* Initialize the PAPI library and get the number of counters available */  
if ((num_hwcntrs = PAPI_num_counters()) <= PAPI_OK)  
    handle_error(1);
```

```
/* Start counting events */  
if (PAPI_start_counters(Events, num_hwcntrs) != PAPI_OK)  
    handle_error(1);
```

Compile-time/Run-time Instrumentation

- Code recompilation
 - -L<path-to-lib> -l<libname>
- Code recompilation not required
 - LD_PRELOAD, LD_LIBRARY_PATH..

Some Factors

Scalability

- Large number of processes
- Many functions
- Different inputs

Data Format

- Some common log formats
- Most are tool-specific

Presentation

- Present data in a hierarchical fashion
- Prioritize – present first what happens to be important

THE TAU PARALLEL PERFORMANCE SYSTEM

Sameer S. Shende

Allen D. Malony

Tuning and Analysis Utilities (TAU)

- Supports tracing and profiling
- Supports different automated instrumentation options
 - Source code
 - Compiler based
- Uses call-path profiling to determine relevant routines for tracing
- Data management framework
 - Relational database
 - Graphical display
- Search traces to automatically identify inefficiency

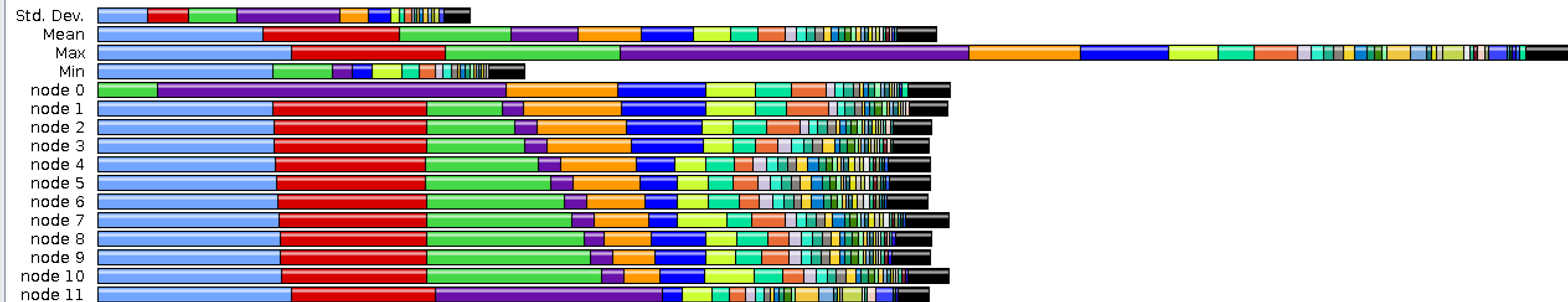
TAU Profiling

- Performance measurements for functions, basic blocks, statements.
- Exclusive and inclusive time spent in functions (ns)
- #times each function was called, mean inclusive time per call, etc.

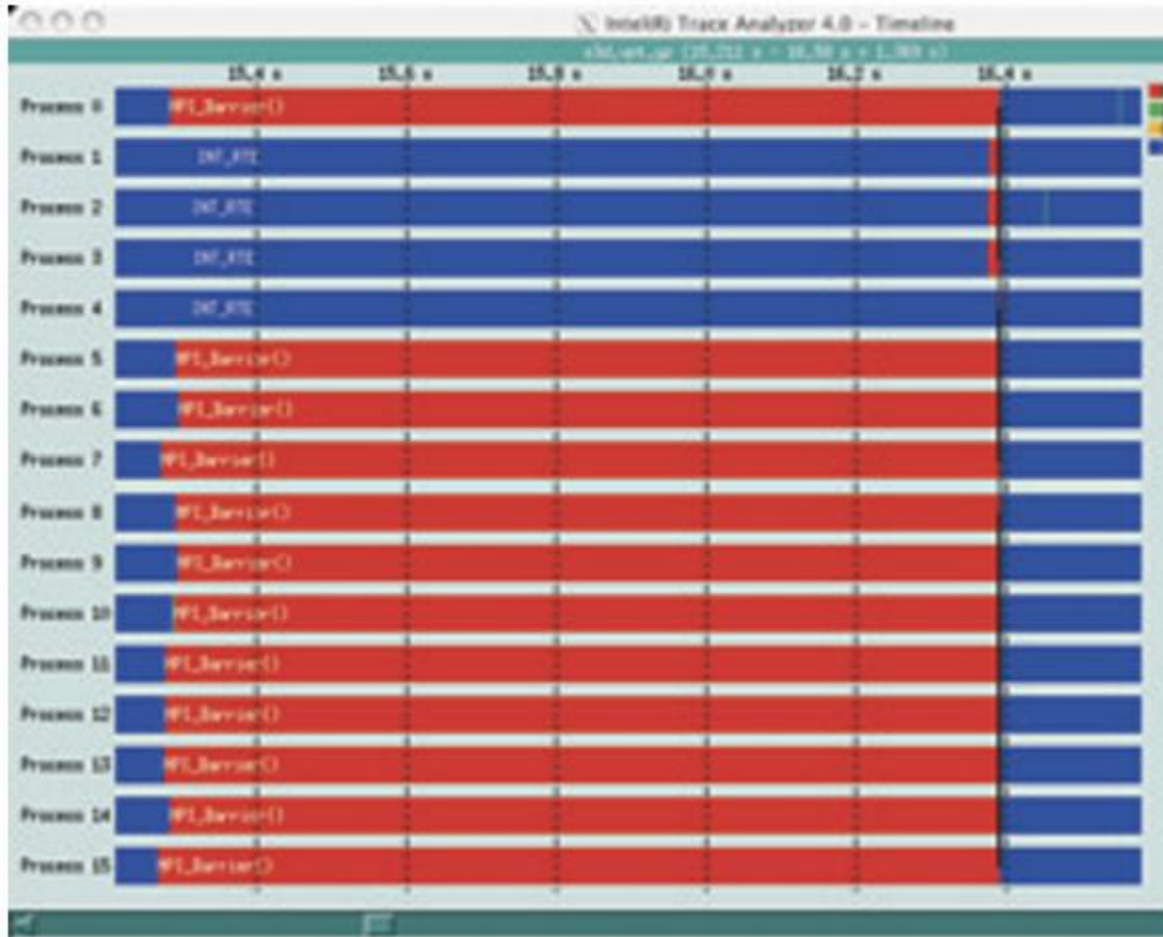
TAU Snapshot

File Options Windows Help

Metric: TIME
Value: Exclusive

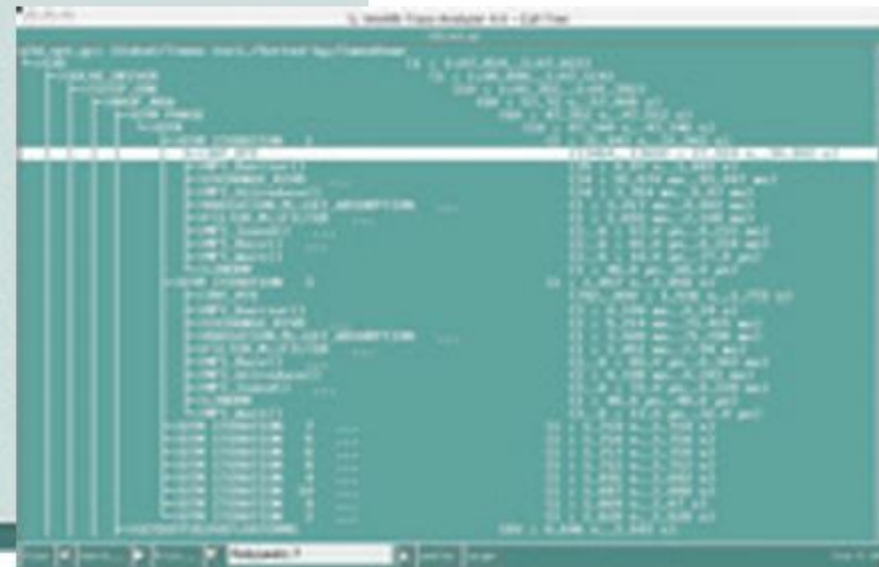


TAU Trace Visualization of S3D using Vampir



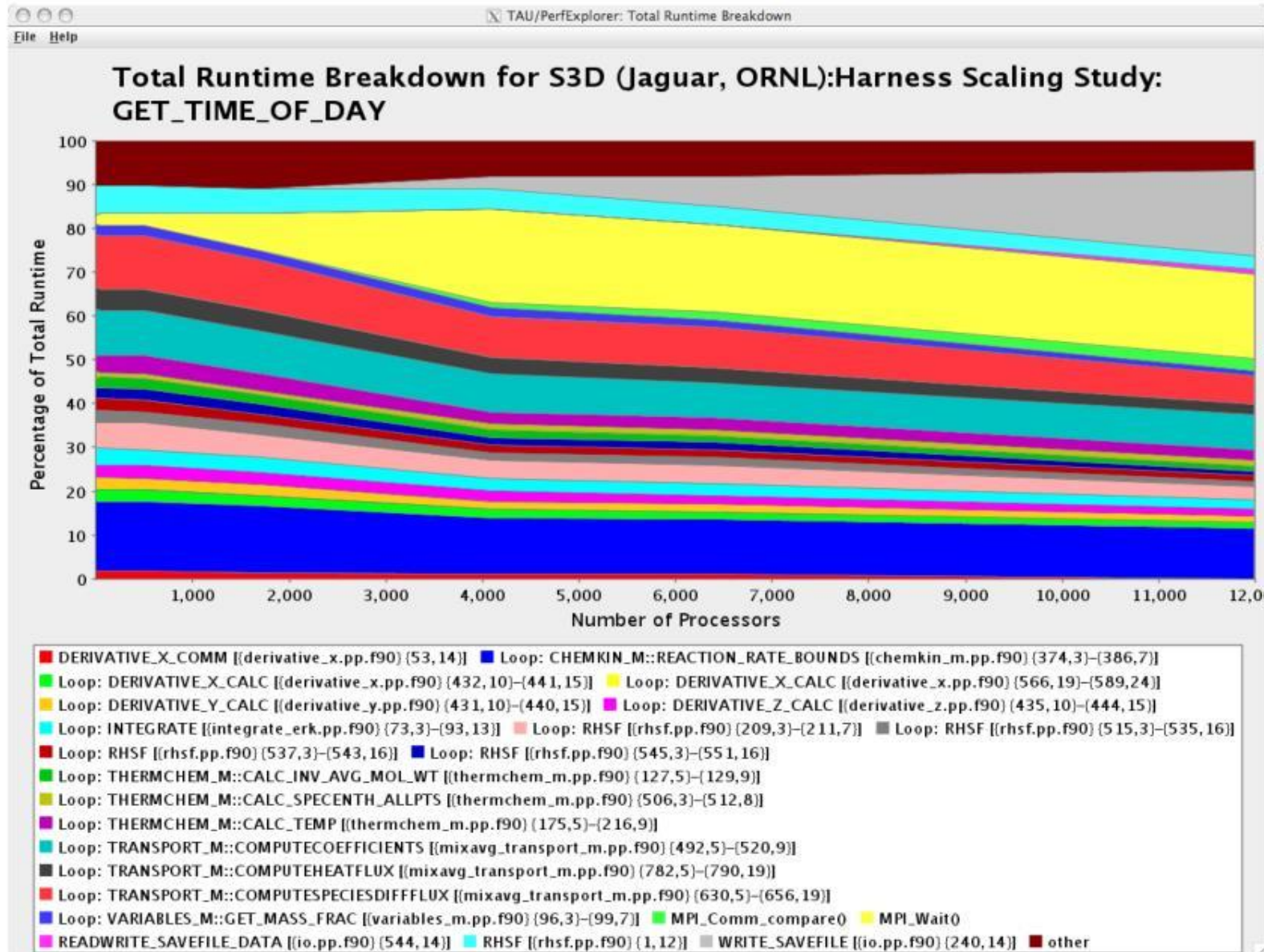
Communication Matrix

Colors represent the message size (communication volume)



Top-down view of function calls (includes inclusive and exclusive times)

PerfExplorer: S3D Total Runtime Breakdown



A different visualization of time spent in computation and communication functions.

Note its easy to comprehend even for 12K processes using this view.

HPCTOOLKIT: Tools for performance analysis of optimized parallel programs

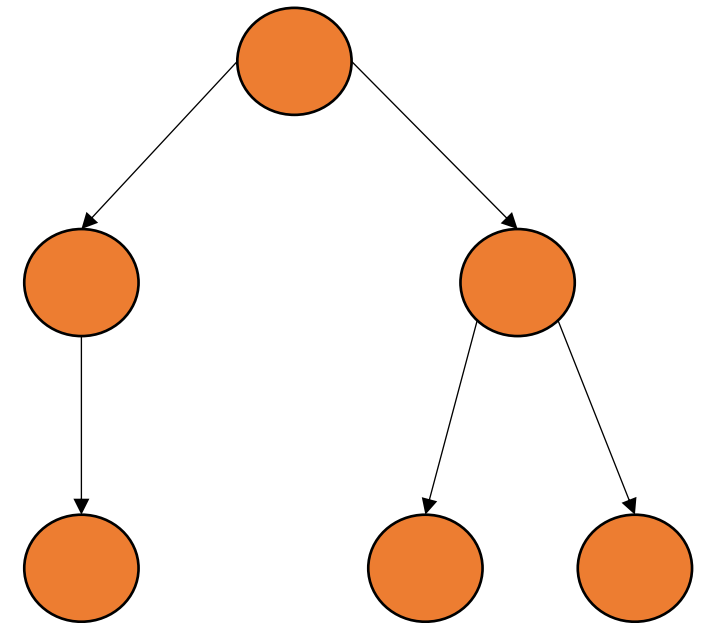
L. Adhianto, S. Banerjee, M. Fagan, M. Krentel, G.
Marin, J. Mellor-Crummey , N. R. Tallent

HPCToolkit Overview

- Avoids instrumentation
 - gprof dilates by 82%
 - Intel's Vtune dilates by 31x
- Uses statistical sampling
- Supports call path profiling: attribute cost incurred to different contexts in which a procedure is called
- Supports collection, correlation of multiple performance metrics

Calling Context Tree

- Each vertex in a tree represents single procedure activation
 - Metrics recorded for that invocation
- Sample count recorded at each node
- Tracing
 - Record the timestamp as well
 - Space-consuming



Analysis

Correlation of performance metrics

- Mapping between object code and source code (“recovering program structure”)

Compute useful metrics

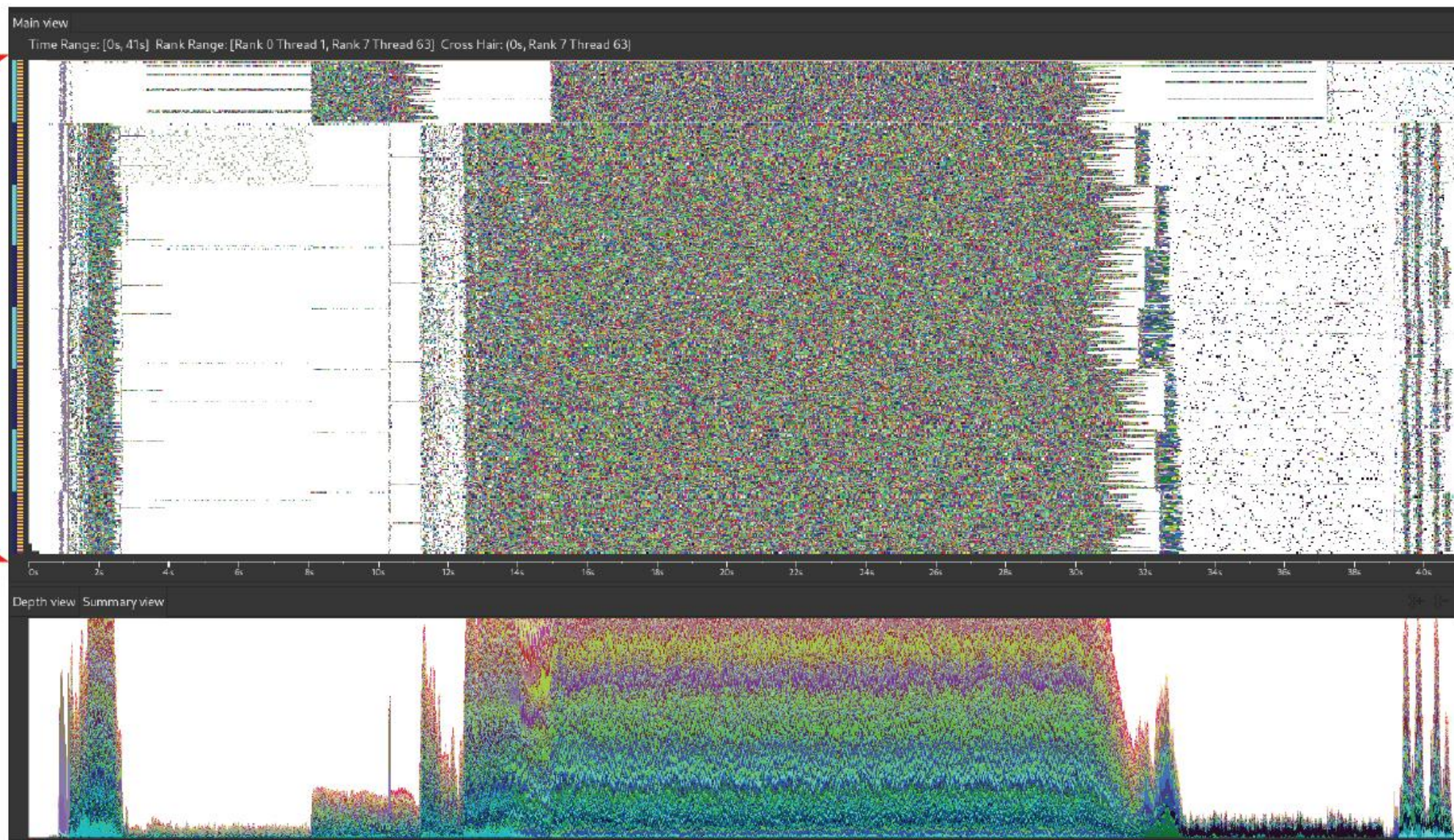
- Knowing where the most cycles are spent
- Derived metrics

Scalability

- Combine call path profiles with program structure
- Estimate idleness, parallel efficiency etc.

hpcprof-mpi: Analyze Measurements of LAMMPS @ 2K threads + 2K GPUs

Analysis on 8 nodes
using 504 threads!



Completes in 41s!

TensorFlow Application

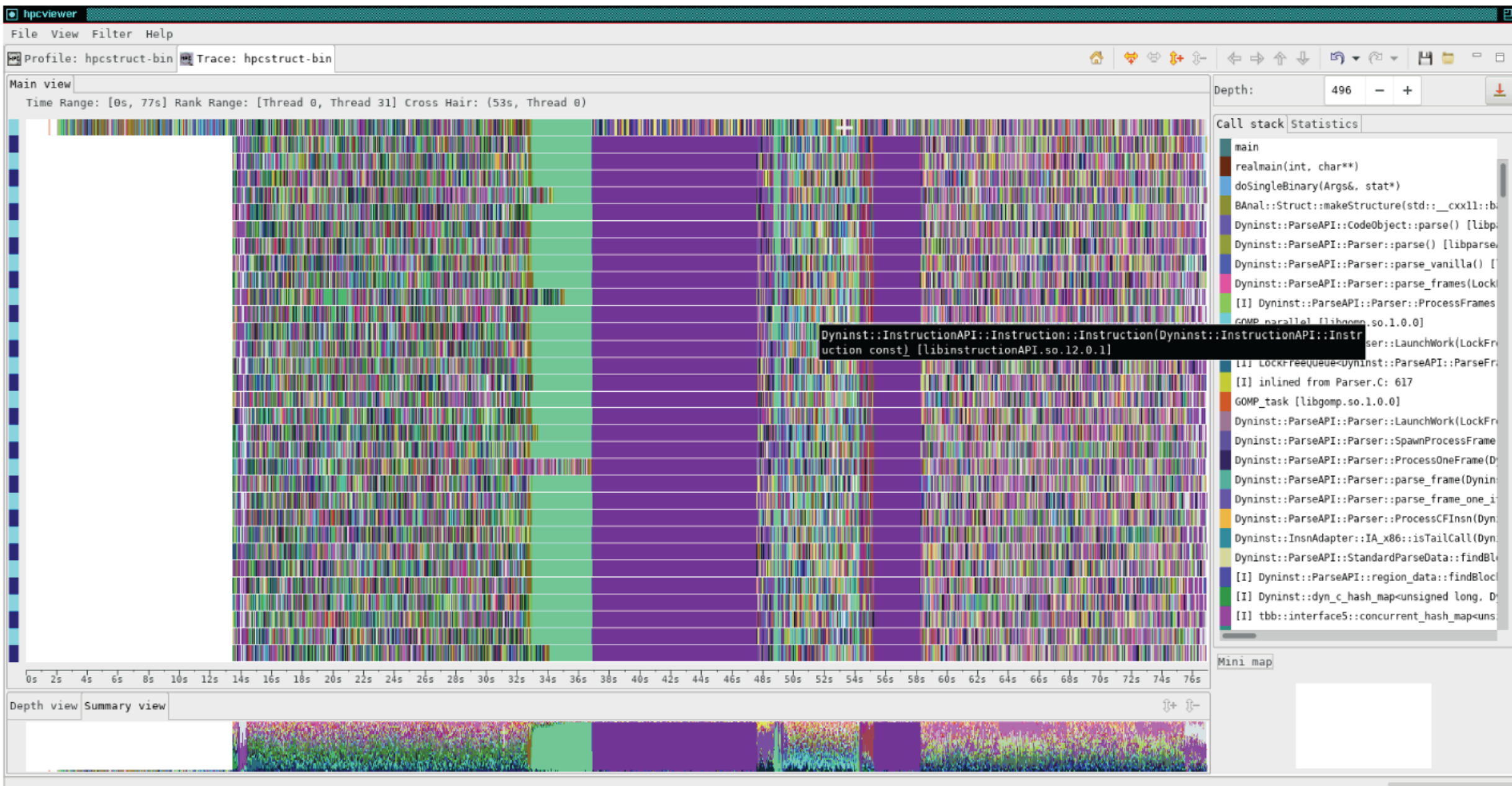
Python: User + TensorFlow

C/C++: TensorFlow

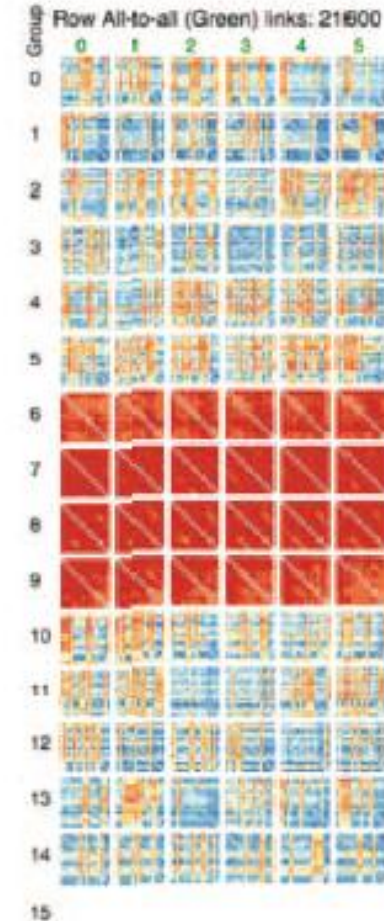
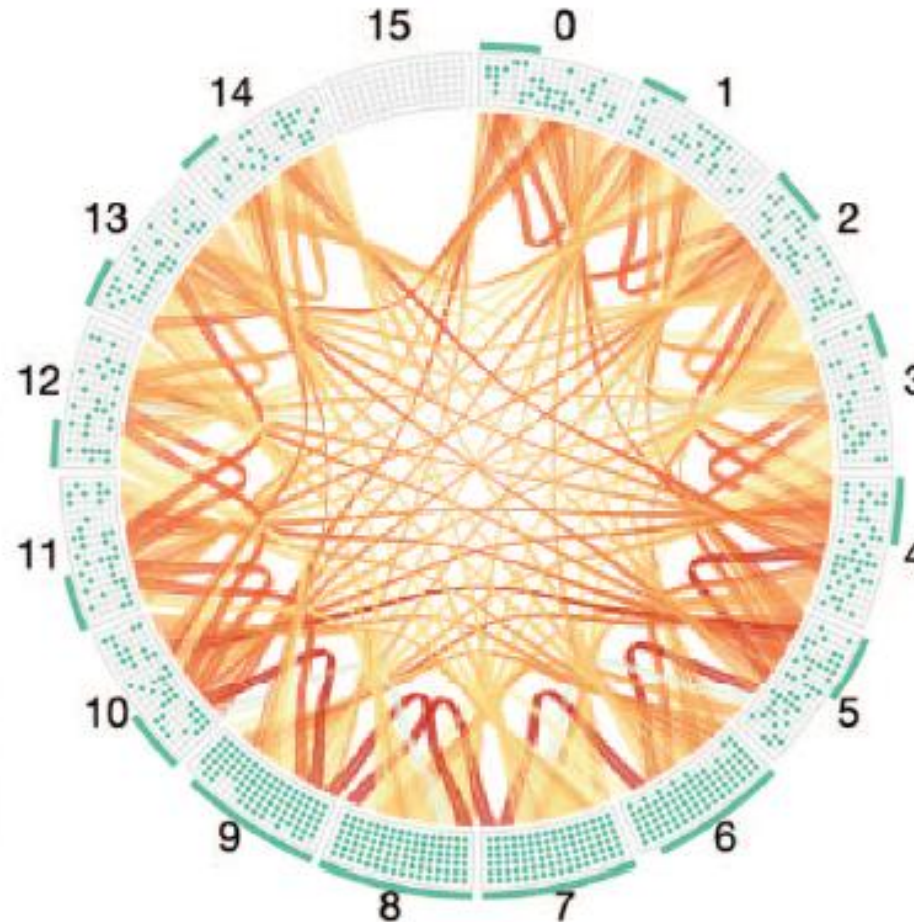
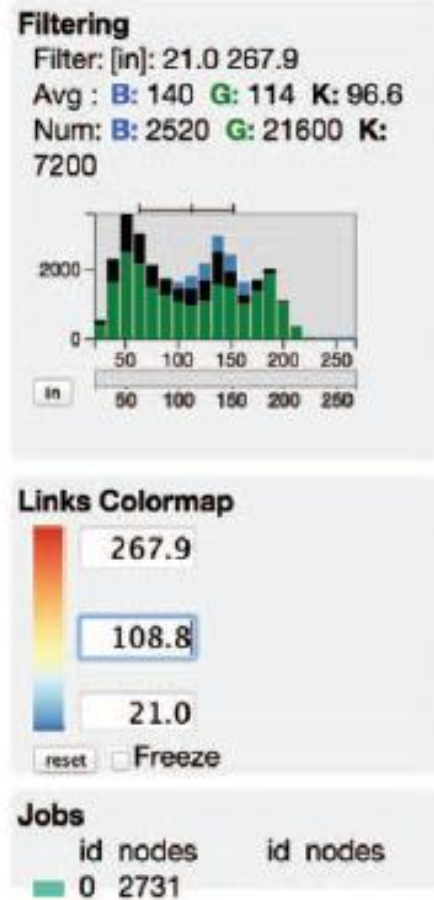
C/C++: Intel TF Extension

SYCL: GPU kernel launch

hpcstruct Example: Analyze 7.7GB TensorFlow library (170MB text) in 77s



System HotSpot Analysis Example



(b) 4D Stencil

Hands-on

- Login to PARAM Rudra
- `ssh -p4422 iitk XY@paramrudra.cdacdelhi.in`
- `cp -r /home/iitk/15-04 $HOME/`
- `cd $HOME/15-04`

Current Content of /home/iitk/15-04

All required files are present

/home/iitk/15-04/exp.sh

/home/iitk/15-04/bin

ipm_key_mpi ipm_parse pl

/home/iitk/15-04/lib

libipm.a libipmf.a libipmf.la libipmf.so libipmf.so.0 libipmf.so.0.0.0 libipm.la libipm.so libipm.so.0
libipm.so.0.0.0

/home/iitk/15-04/NPB (*executables and submission script samples*)

cg.C.x cg.D.x is.D.x is.E.x lu.C.x lu.D.x lu.E.x mg.D.x mg.E.x
subCG-D-P32.sh subCG-D-P64.sh subLU-D-P32.sh subLU-D-P64.sh

NAS Parallel Benchmarks

<https://www.nas.nasa.gov/publications/npb.html>

- Five kernels
 - IS - Integer Sort, random memory access
 - EP - Embarrassingly Parallel
 - CG - Conjugate Gradient, irregular memory access and communication
 - MG - Multi-Grid on a sequence of meshes, long- and short-distance communication, memory intensive
 - FT - discrete 3D fast Fourier Transform, all-to-all communication
- Three pseudo applications
 - BT - Block Tri-diagonal solver
 - SP - Scalar Penta-diagonal solver
 - LU - Lower-Upper Gauss-Seidel solver

Profiling using IPM

- Integrated Performance Monitoring (IPM) profiler

<http://ipm-hpc.sourceforge.net/userguide.html>

Compile IPM

// already done

- Directory structure after compilation

bin etc lib

Link with IPM

- Edit Makefile of NPB (-L<library path> -lipm) *// already done*
- Compile application (NPB) *// already done*
- export LD_LIBRARY_PATH=<library path>:\$LD_LIBRARY_PATH
 - See next slide

Export Environment Variables

- cat exp.sh

Run the below commands (present in exp.sh)

- export LD_LIBRARY_PATH=\$HOME/15-04/lib:\$LD_LIBRARY_PATH

// Ensure that =\$HOME/15-04/lib contains the libipm files*

- export IPM_REPORT=full
- export IPM_LOG=full
- export IPM_REPORT_MEM=yes

Next Steps

- cd NPB
- ls sub*sh
 - Some sample batch scripts
- ls *.x
 - Sample NPB executables (compiled with IPM)
 - You may use LD_PRELOAD for runtime linking *// not required now*
- Submit job
 - sbatch subCG-D-P64.sh (*one of the submission scripts in NPB directory*)
 - Open the script and check the executable name, ensure its present in the current directory

```

CS633 $ cp -r /home/iitk/15-04 .
CS633 $ ls
15-04
CS633 $ cd 15-04/
CS633 $ ls
NPB  bin  exp.sh  lib
CS633 $ cat exp.sh
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HOME/15-04/lib
export IPM_REPORT=full
export IPM_LOG=full
export IPM_REPORT_MEM=yes
CS633 $ export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HOME/15-04/lib
CS633 $ export IPM_REPORT=full
CS633 $ export IPM_LOG=full
CS633 $ export IPM_REPORT_MEM=yes
CS633 $ cd NPB/
CS633 $ ls
cg.D.x  lu.C.x  lu.D.x  lu.E.x  subCG-D-P32.sh  subCG-D-P64.sh  subLU-D-P32.sh
CS633 $ sbatch subLU-D-P32.sh
Submitted batch job 21554
CS633 $ squeue --me

```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
21554	cpu	LU_D	iitk_	R	0:06	1	rdcn25

Once the job finishes, there should be an .xml file in the same folder from where the job was submitted
 Ensure .err is empty

Typical Time Required for LU (1 run)

Strong Scaling LU on 32, 48, 64, 96

- LU_D_21554.out: Time in seconds = 468.49
- LU_D_21557.out: Time in seconds = 303.79
- LU_D_21585.out: Time in seconds = 228.16
- LU_D_21597.out: Time in seconds = 156.60

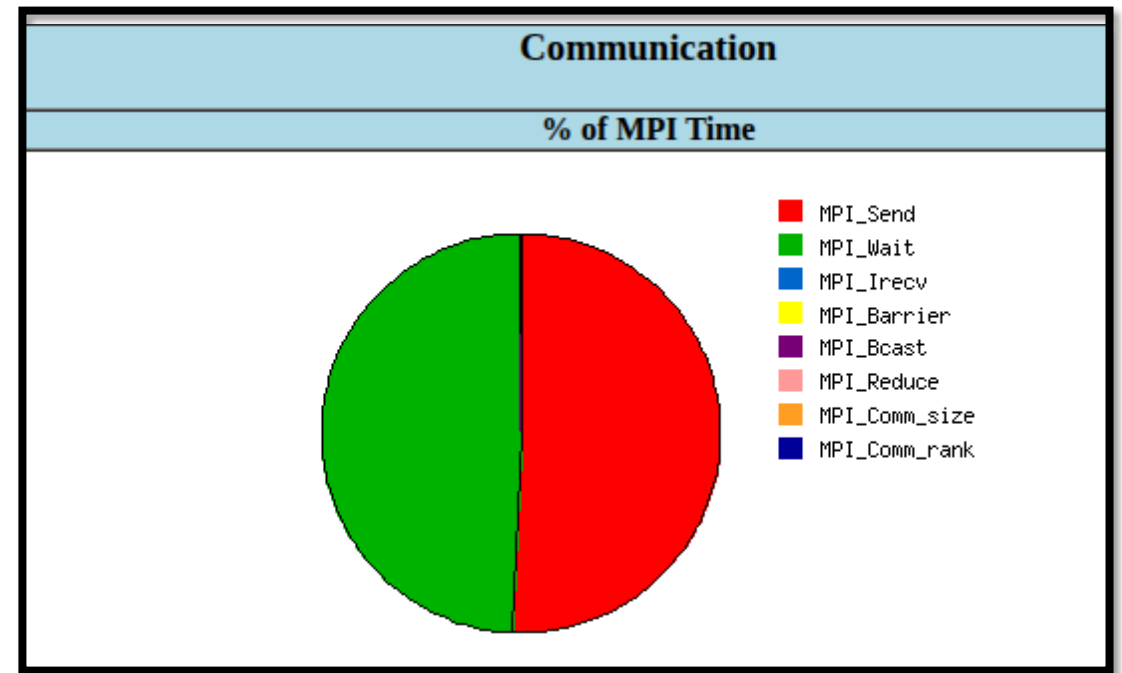
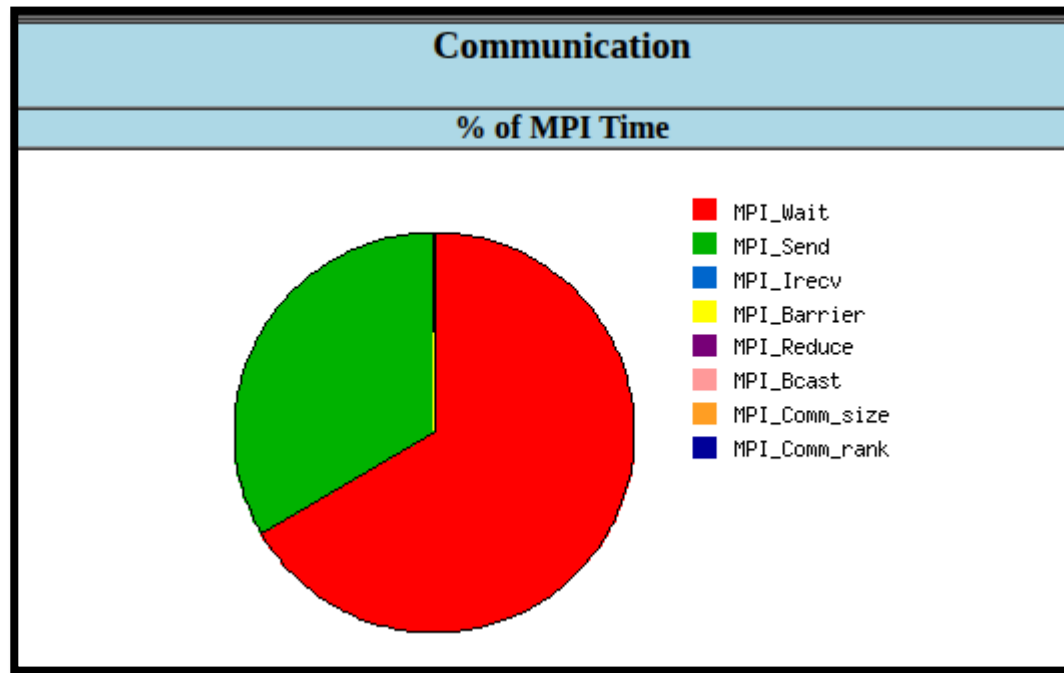
Using IPM on your laptop

- `cp *.xml bin/`
- Transfer the xml to your local system (*Note: run this command from your laptop/desktop, this copies the bin folder to your laptop.*)
 - `scp -P 4422 -r iitk_XY@paramrudra.cdacdelhi.in:~/15-04/bin .`
- Required files are: `ipm_parse`, `ipm_key_mpi` and `pl` (ploticus)
- Optional: Install ploticus in your local system *// this is already done, you will find pl file in bin directory, if that does not work, please install*
- `export IPM_KEYFILE=ipm_key_mpi`
- `export PATH=.:$PATH` *// this ensures pl is in the PATH environment variable*
- `./ipm_parse -html <xml file name>`
 - should generate a folder with `index.html` inside a folder

Visual Profiling

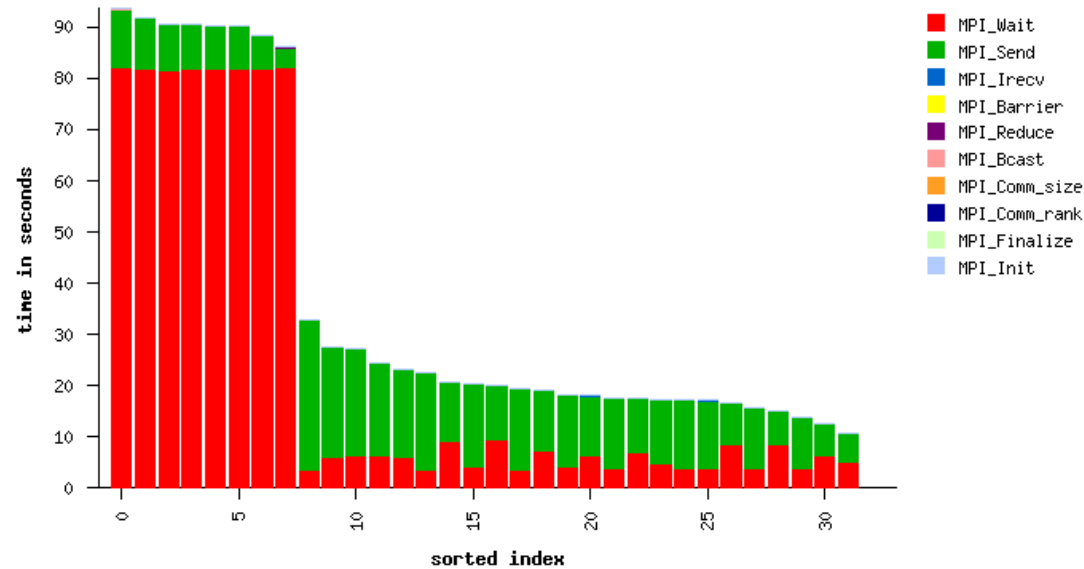
- Open index.html for both runs in your browser (separate tabs)
- Analyze the results

CG, Class D problem size, P=32 vs. 64 (ppn=32)



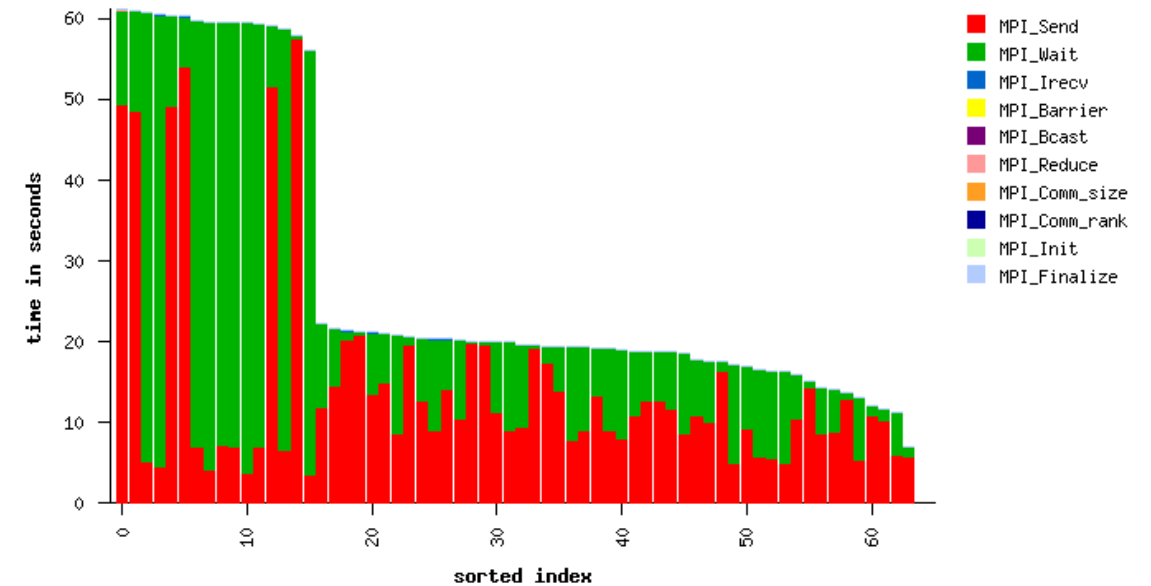
CG, Class D problem size, P=32 vs. 64 (ppn=32)

Communication balance by task (sorted by MPI time)



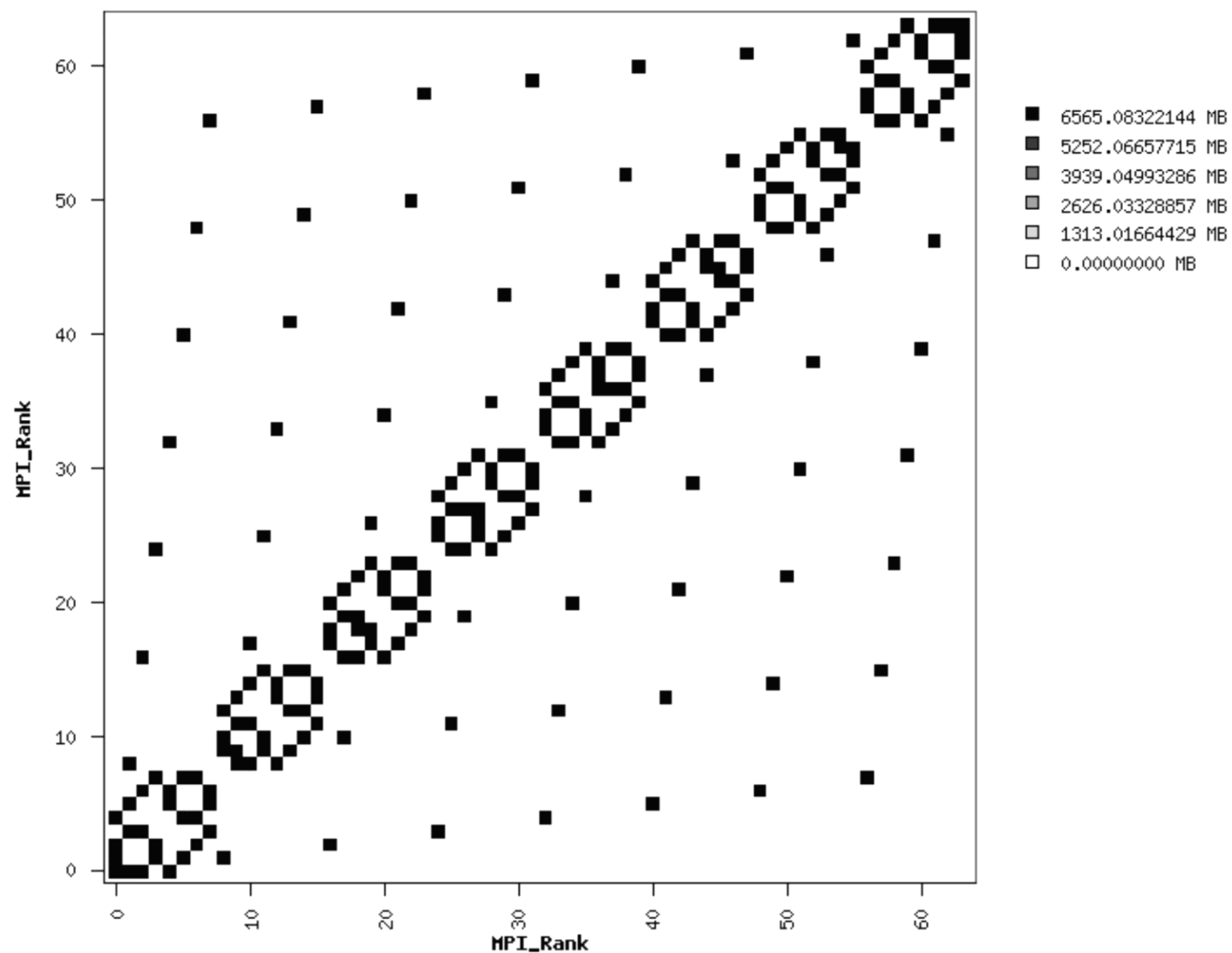
[by MPI rank](#), [time detail by MPI time](#), [time detail by rank](#), [call list](#)

Communication balance by task (sorted by MPI time)



[by MPI rank](#), [time detail by MPI time](#), [time detail by rank](#), [call list](#)

Communication Topology : point to point data flow



Memory Consumption on 2 Nodes

